📘 MODULE 13: JOIN / APPEND / APPENDCOLS / TRANSACTION

13.1 Concept

These commands **combine data from different searches or events**. They are convenient but **expensive**, and Splunk usually has a better way.

13.2 join: add fields from a second search, matched on a common field

spl

```
index=_internal sourcetype=splunkd_access
| stats count AS requests by clientip
| join type=left clientip
  [ search index=_internal sourcetype=splunkd_access status>=400
   | stats count AS errors by clientip ]
```

What it does: for each IP, adds the error count from the second search.
Types: inner (default; keep only matches) and left (keep all left rows; outer is the same).
Problems: the second search is a subsearch, so it has **limits (about 50,000 rows and 60 seconds by default)**, it runs separately, and it is slow.

The better way (stats, one pass):

spl

```
index=_internal sourcetype=splunkd_access
| eval is_error=if(status>=400,1,0)
| stats count AS requests, sum(is_error) AS errors by clientip
```

Same result, one search, no limits. **Rule: if both datasets can be fetched in one base search, use stats.**

Combining two different indexes with stats:

spl

```
(index=auth) OR (index=vpn)
| eval user=coalesce(user, vpn_user)
| stats values(index) AS sources, count by user
```

13.3 append: stack results from a second search under the first

spl

index=_internal sourcetype=splunkd | stats count AS events by host

| append [search index=_internal sourcetype=splunkd_access | stats count AS web_events by host]

| stats values(events) AS events, values(web_events) AS web_events by host

append adds **rows** at the bottom. You often need a final stats to merge rows with the same key. It has the same subsearch limits.

13.4 appendcols: add columns side by side

spl

... | stats count AS total

| appendcols [search ... | stats count AS errors]

Danger: it matches by **row position, not by a key**, so if rows differ in order or number, data is misaligned. Use it only for single-row results. Otherwise use stats or join.

13.5 transaction: group related events into one

spl

index=_internal sourcetype=splunkd_access

| transaction clientip maxspan=5m maxpause=1m

| where eventcount>5

| table _time clientip duration eventcount

What it does: groups events from the same clientip into a “session”: at most 5 minutes total (maxspan) with no gap longer than 1 minute (maxpause).
Output fields: duration (seconds), eventcount, and the combined _raw.
Other options: startswith="login" and endswith="logout" define the session boundaries.
Problems: memory-heavy, slow on big data, and it can’t be distributed well. It is **last resort**.

The stats alternative for session-like analysis:

spl

index=_internal sourcetype=splunkd_access

| stats min(_time) AS start, max(_time) AS end, count BY clientip

| eval duration=end-start

Use transaction only when you truly need gap-based or start/end-marker session splitting.

13.6 Decision table

Need	Use
Combine fields from multiple sources by a key	stats (first choice) or lookup
Add details from a small, second search	join (small data only)
Combine result sets (stack)	append + stats
Calculate one-row metrics side by side	appendcols (rare)
Session with start/end markers or time gaps	transaction
Group events by ID, no gap logic	stats

13.7 Hands-on

1. Rewrite the join example above using stats and confirm the same numbers.
2. Run transaction on clientip and compare the run time with the stats version (look at the job inspector, under Job → Inspect Job).
3. Use append to combine counts by host from two sourcetypes.

13.8 Troubleshooting

Symptom	Cause
join returns partial data	Subsearch limit reached
join returns nothing	Key field names or values don't match (case, spaces)
Misaligned appendcols	Matches by row, not key
transaction very slow	Too much data, so narrow it or switch to stats

13.9 Interview Q&A

- **Why avoid join?** Subsearch limits, slow, and stats does the same in one pass.
- **append vs appendcols?** Append adds rows, appendcols adds columns by position.
- **When use transaction?** For session grouping with start/end markers or time-gap logic.

- **transaction vs stats?** stats is faster and scalable, transaction is flexible but expensive.
- **What does maxspan do?** Caps the total time between the first and last event in a transaction.

13.10 Revision

Prefer stats · join has subsearch limits · append stacks rows · appendcols aligns by position · transaction = last resort.